Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92301733057**

Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:
Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:
Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are sonic of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:
Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly actually be converted to troublinstead of trouble because the ends were just chopped off (ughh, how crude!). There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porters Algorithm. Here is an example of stemming in action with Porter Stemmer:

	original word	stemmed words
0	connect	connect
1	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

Stopword Removal

Stop words are a set of commonly used words in a language. Examples of stop words in English are "a", ‘‘the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?", you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

Colab link: https://colab.research.google.com/drive/1xXMlj9wSvTQxIlVjFR9uwwIjQEuAFTA9?usp=sharing

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92301733057**

Program (Code):

To be attached with

1. Tokenization

```
text = "Hi there! I am Payal Makwana, Software Engineer working at Google."
tokens = word_tokenize(text)
print("\n1. Original Tokens:\n", tokens)
```

```
1. Original Tokens:
 ['Hi', 'there', '!', 'I', 'am', 'Payal', 'Makwana', ',', 'Software', 'Engineer', 'working',
'at', 'Google', '.']
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

2. Filteration

```
filtered_tokens = [word for word in tokens if word not in string.punctuation]
print("\n2. Filtered Tokens:\n", filtered_tokens)
```

```
2. Filtered Tokens:
 ['Hi', 'there', 'I', 'am', 'Payal', 'Makwana', 'Software', 'Engineer', 'working', 'at',
'Google']
```

3. StopWords Removal

```
from nltk.corpus import stopwords
import nltk
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
clean_tokens = [word for word in filtered_tokens if word.lower() not in stop_words]
print("\n3. After Stopword Removal:\n", clean_tokens)
```

```
...
3. After Stopword Removal:
 ['Hi', 'Payal', 'Makwana', 'Software', 'Engineer', 'working', 'Google']
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92301733057**

4. PoS Tagging

```
import nltk
nltk.download('averaged_perceptron_tagger_eng')
from nltk import pos_tag

pos_tags = pos_tag(clean_tokens)
print("\n4. POS Tags:\n", pos_tags)
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.

4. POS Tags:
 [('Hi', 'NNP'), ('Payal', 'NNP'), ('Makwana', 'NNP'), ('Software', 'NNP'), ('Engineer', 'NNP'), ('working', 'VBG'), ('Google', 'NNP')]
```

5. Noun Phrase Chunking

```
from nltk.chunk import RegexpParser
grammar = "NP: {<DT>?<JJ><NN.>+}"
chunk_parser = RegexpParser(grammar)
chunk_tree = chunk_parser.parse(pos_tags)

print("\n5. Noun Phrase Chunking:\n")
print(chunk_tree)
```

```
5. Noun Phrase Chunking:

(S
  Hi/NNP
  Payal/NNP
  Makwana/NNP
  Software/NNP
  Engineer/NNP
  working/VBG
  Google/NNP)
```

6. Dependancy Parsing

```
doc = nlp(text)

print("\n6. Dependency Parsing:\n")
```

```
for token in doc:
    print(f"{token.text} --> {token.dep_} --> {token.head.text}")
```

```
6. Dependency Parsing:

Hi --> ROOT --> Hi
there --> advmod --> Hi
! --> punct --> Hi
I --> nsubj --> am
am --> ROOT --> am
Payal --> compound --> Makwana
Makwana --> attr --> am
, --> punct --> Makwana
Software --> compound --> Engineer
Engineer --> nsubj --> working
working --> acl --> Makwana
at --> prep --> working
Google --> pobj --> at
. --> punct --> am
```

Observation:

Text preprocessing is used to clean and prepare textual data for analysis. Tokenization divides the text into smaller units such as words or sentences. Stop word removal eliminates commonly used words like “is” and “the” that do not add significant meaning. Stemming reduces words to their root form, simplifying the data and making it easier to process.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92301733057**

Post Lab Exercise:

1. Take your own document of 10 sentences and perform the same tasks. Attach code and screenshot of your output.

```
# Take your own document of 10 sentences and perform the same tasks. Attach code and
screenshot of your output.
import nltk
import string
import spacy

# Download required data
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag, RegexpParser
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Load spacy
nlp = spacy.load("en_core_web_sm")

# INPUT DOCUMENT (10 sentences)
text = """I love eating pizza on weekends.
The weather is very hot today.
She is going to college by bus.
This mobile phone is very expensive.
I completed my project yesterday.
He plays cricket every evening.
The movie was really interesting.
My friend lives in another city.
We are learning programming together.
The food in this restaurant is delicious."""

print("Original Text:\n", text)

# TOKENIZATION
```

```
tokens = word_tokenize(text)
print("\nTokens:\n", tokens)

# FILTERING
filtered = [w for w in tokens if w not in string.punctuation]

# STOPWORDS REMOVAL
stop_words = set(stopwords.words('english'))
clean_tokens = [w for w in filtered if w.lower() not in stop_words]
print("\nClean Tokens:\n", clean_tokens)

# POS TAGGING
pos_tags = pos_tag(clean_tokens)
print("\nPOS Tags:\n", pos_tags)

# NOUN PHRASE CHUNKING
grammar = "NP: {<DT>?<JJ>*<NN.*>+}"
cp = RegexpParser(grammar)
tree = cp.parse(pos_tags)
print("\nChunk Tree:\n", tree)

# DEPENDENCY PARSING
doc = nlp(text)
print("\nDependency Parsing:\n")
for token in doc:
    print(token.text, "->", token.dep_, "->", token.head.text)

# STEMMING
stemmer = PorterStemmer()
stemmed = [stemmer.stem(word) for word in clean_tokens]

print("\nStemming:\n", stemmed)

# LEMMATIZATION
lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(word) for word in clean_tokens]

print("\nLemmatization:\n", lemmatized)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92301733057**

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
Original Text:
 I love eating pizza on weekends.
The weather is very hot today.
She is going to college by bus.
This mobile phone is very expensive.
I completed my project yesterday.
He plays cricket every evening.
The movie was really interesting.
My friend lives in another city.
We are learning programming together.
The food in this restaurant is delicious.

Tokens:
 ['I', 'love', 'eating', 'pizza', 'on', 'weekends', '.', 'The', 'weather', 'is', 'very', 'hot', 'today', '.', 'She', 'is', 'going', 'to',
     'college', 'by', 'bus', '.', 'This', 'mobile', 'phone', 'is', 'very', 'expensive', '.', 'I', 'completed', 'my', 'project',
     'yesterday', '.', 'He', 'plays', 'cricket', 'every', 'evening', '.', 'The', 'movie', 'was', 'really', 'interesting', '.', 'My',
     'friend', 'lives', 'in', 'another', 'city', '.', 'We', 'are', 'learning', 'programming', 'together', '.', 'The', 'food', 'in', 'this',
     'restaurant', 'is', 'delicious', '.']

Clean Tokens:
 ['love', 'eating', 'pizza', 'weekends', 'weather', 'hot', 'today', 'going', 'college', 'bus', 'mobile', 'phone', 'expensive',
     'completed', 'project', 'yesterday', 'plays', 'cricket', 'every', 'evening', 'movie', 'really', 'interesting', 'friend', 'lives',
     'another', 'city', 'learning', 'programming', 'together', 'food', 'restaurant', 'delicious']

POS Tags:
 [('love', 'VB'), ('eating', 'VBG'), ('pizza', 'JJ'), ('weekends', 'NNS'), ('weather', 'RBR'), ('hot', 'JJ'), ('today', 'NN'),
```

('going', 'VBG'), ('college', 'NN'), ('bus', 'NN'), ('mobile', 'IN'), ('phone', 'NN'), ('expensive', 'JJ'), ('completed', 'VBN'), ('project', 'NN'), ('yesterday', 'NN'), ('plays', 'VBZ'), ('cricket', 'NN'), ('every', 'DT'), ('evening', 'NN'), ('movie', 'NN'), ('really', 'RB'), ('interesting', 'JJ'), ('friend', 'NN'), ('lives', 'VBZ'), ('another', 'DT'), ('city', 'NN'), ('learning', 'VBG'), ('programming', 'VBG'), ('together', 'RB'), ('food', 'NN'), ('restaurant', 'NN'), ('delicious', 'JJ')]

Chunk Tree:
(S
love/VB
eating/VBG
(NP pizza/JJ weekends/NNS)
weather/RBR
(NP hot/JJ today/NN)
going/VBG
(NP college/NN bus/NN)
mobile/IN
(NP phone/NN)
expensive/JJ
completed/VBN
(NP project/NN yesterday/NN)
plays/VBZ
(NP cricket/NN)
(NP every/DT evening/NN movie/NN)
really/RB
(NP interesting/JJ friend/NN)
lives/VBZ
(NP another/DT city/NN)
learning/VBG
programming/VBG
together/RB
(NP food/NN restaurant/NN)
delicious/JJ)

Dependency Parsing:

I -> nsubj -> love
love -> ROOT -> love
eating -> xcomp -> love
pizza -> dobj -> eating
on -> prep -> pizza
weekends -> pobj -> on
. -> punct -> love

-> dep -> .
The -> det -> weather
weather -> nsubj -> is
is -> ROOT -> is
very -> advmod -> hot
hot -> acomp -> is
today -> npadvmod -> is
. -> punct -> is

-> dep -> .
She -> nsubj -> going
is -> aux -> going
going -> ROOT -> going
to -> prep -> going
college -> pobj -> to
by -> prep -> going
bus -> pobj -> by
. -> punct -> going

-> dep -> .
This -> det -> phone
mobile -> amod -> phone
phone -> nsubj -> is
is -> ROOT -> is
very -> advmod -> expensive
expensive -> acomp -> is
. -> punct -> is

-> dep -> .
I -> nsubj -> completed
completed -> ROOT -> completed
my -> poss -> project
project -> dobj -> completed
yesterday -> npadvmod -> completed
. -> punct -> completed

-> dep -> .
He -> nsubj -> plays
plays -> ROOT -> plays
cricket -> dobj -> plays
every -> det -> evening
evening -> npadvmod -> plays
. -> punct -> plays

-> dep -> .
The -> det -> movie
movie -> nsubj -> was
was -> ROOT -> was
really -> advmod -> interesting
interesting -> acomp -> was
. -> punct -> was

-> dep -> .
My -> poss -> friend
friend -> nsubj -> lives
lives -> ROOT -> lives
in -> prep -> lives
another -> det -> city
city -> pobj -> in
. -> punct -> lives

-> dep -> .
We -> nsubj -> learning
are -> aux -> learning
learning -> ROOT -> learning
programming -> xcomp -> learning
together -> advmod -> programming
. -> punct -> learning

-> dep -> .
The -> det -> food
food -> nsubj -> is
in -> prep -> food
this -> det -> restaurant
restaurant -> pobj -> in
is -> ROOT -> is
delicious -> acomp -> is
. -> punct -> is

Stemming:
['love', 'eat', 'pizza', 'weekend', 'weather', 'hot', 'today', 'go', 'colleg', 'bu', 'mobil', 'phone', 'expens', 'complet', 'project', 'yesterday', 'play', 'cricket', 'everi', 'even', 'movi', 'realli', 'interest', 'friend', 'live', 'anoth', 'citi', 'learn', 'program', 'togeth', 'food', 'restaur', 'delici']

Lemmatization:

2.Write your observation for stemming and lemmatization you obtained for the sentences.

Stemming and lemmatization are techniques used to transform words into their base form. Stemming applies simple rule-based trimming, which can sometimes produce non-meaningful outputs, such as converting “studies” into “studi”. In contrast, lemmatization takes into account the context and meaning of the word, converting it into a valid dictionary form, such as “studies” to “study”. Hence, stemming is faster but less precise, whereas lemmatization is more accurate and generates meaningful words.